COMP90077

Lecture 2: Matroids

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Student Reps and Feedback

- Rong Shi
- Charles Kyn Nierves
- Let me/Xiao/reps know ASAP if there's something I can improve
- Possibly some data structures (streaming algorithms) at the end:
  - Examples: Determining majority in an election

Lecture 1 and Tutorial 1

- Proofs more tedious than expected
- Today: Better proof for optimality of Kruskal
- Why proofs?
  - Learn problem structure
  - Passing test cases ≠ correctness on all inputs
  - Proof of correctness = correctness on all inputs
- Why notation and terminology?
  - Unambiguous and concise. DRY (Don't Repeat Yourself)
  - Example: "spanning tree" vs "subgraph that connects every vertex to every other vertex and has no cycles"
  - See also: List comprehensions in Python
  - Too much notation and terminology also bad

Combinatorial Optimization

- Goal: Discover general principles of algorithms and problems MST
- Def. A *combinatorial optimization problem* consists of:
  - Finite set of elements E
  - **(Objective function)** g that maps subsets of E to real numbers
  - **(Feasible sets)** Collection $\mathcal{F}$ of subsets of E
  - **Goal**: Find a feasible subset S that maximizes/minimizes g

$g: 2^E \to \mathbb{R}$

g additive = g sum of weights

- Def. A function g on subsets of E is additive if there is a weight function w on E such that for every subset S, $g(S) = \sum_{e \in S} w(e)$

- Thm. When $\mathcal{F}$ is the set of spanning trees of a graph and g is additive, then greedy finds an optimal solution.

Combinatorial Optimization

- Goal: Discover general principles of algorithms and problems
- Def. A *combinatorial optimization problem* consists of:
  - Finite set of elements E
  - **(Objective function)** g that maps subsets of E to real numbers
  - **(Feasible sets)** Collection $\mathcal{F}$ of subsets of E
  - **Goal**: Find a feasible subset S that maximizes/minimizes g

For what g and $\mathcal{F}$ does a greedy approach work?

Independence Systems

- An independence system $M = (E, \mathcal{I})$ consists of:
  - **(Ground set)** Finite set of elements E
  - **(Independent sets)** Collection of subsets of elements $\mathcal{I}$
- Satisfying:
  - **(Empty set)** The empty set is independent
  - **(Downwards closed)** If S is independent then so is every subset of S

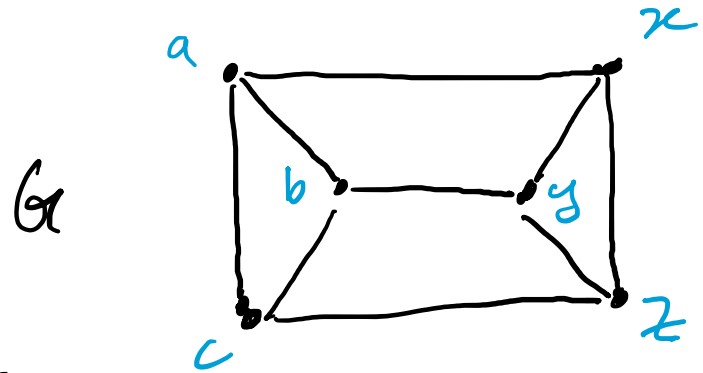
- Many problems have the form "find max-weight independent set"
- **Knapsack problem**: Given items with weights and sizes, and capacity C find max-weight subset with total size at most C
- What other problems have this form? "Packing" type problems

Matroids

- An *independence system* $M = (E, \mathcal{I})$ consists of:
  - **(Ground set)** Finite set of elements E
  - **(Independent sets)** Collection of subsets of elements $\mathcal{I}$
- Satisfying:
  - **(Empty set)** The empty set is independent
  - **(Downwards closed)** If S is independent then so is every subset of S

- A *matroid* $M = (E, \mathcal{I})$ is an independence system that also has:
  - **(Augmentation)** If S, T independent and $|S| > |T|$, then there exists $e \in S \setminus T$ such that $T \cup \{e\}$ is also independent

Graphical Matroids



- Graph $G = (V, E)$
- Graphical matroid $M_G = (E, \mathcal{I})$ consists of:
  - Ground set E (edge set of G)
  - Subset of edges S is independent if it is acyclic
- Matroid properties
  - **(Empty set)** The empty set is independent
  - **(Downwards closed)** If S is independent then so is every subset of S
  - **(Augmentation)** If S, T independent and $|S| > |T|$, then there exists $e \in S \setminus T$ such that $T \cup \{e\}$ is also independent

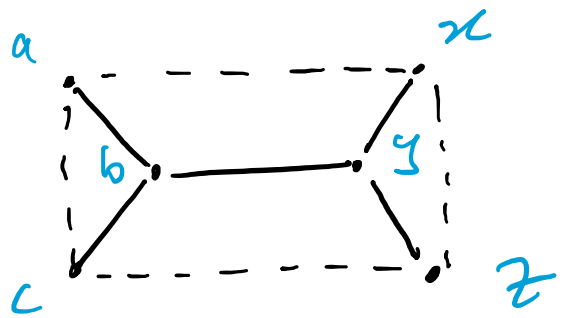
Augmentation for Graphical Matroids

- Lemma. If S, T acyclic and $|S| > |T|$, then there exists $e \in S \setminus T$ such that $T \cup \{e\}$ is also acyclic
- Tutorial 1 Exercise 2 (Solution more complicated than I'd like…)

Proof.

- #Conn-Comp(F) = number of connected components of F (counting isolated vertices)
- #Conn-Comp(empty graph) = n
- F acyclic implies #Conn-Comp(F) = n - |F|
  - Each edge of F reduces connected components by 1
- #Conn-Comp(S) < #Conn-Comp(T)
- Some component of ~~T~~ S contains two vertices in different comps of ~~S~~ T

Bases



- Analog of spanning trees
- Def. An independent set S is a *basis* if it is maximally independent, i.e. there is no proper superset of S that is independent

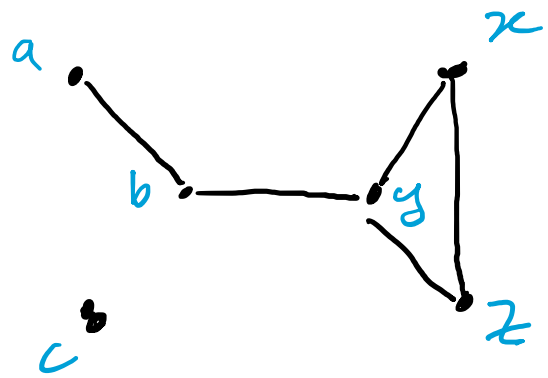
Prop. Every base has the same size (called the *rank* of the matroid).

Why?

Follows from augmentation property

Application: Every spanning tree has the same size.

Circuits



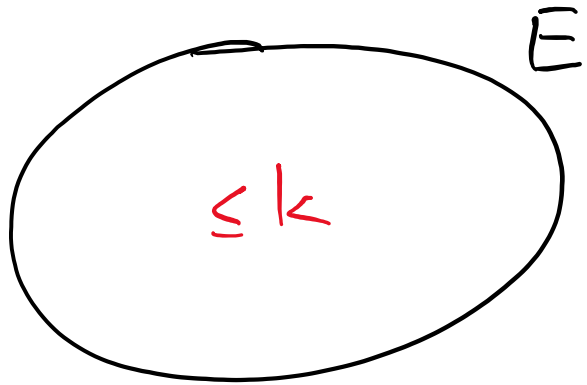
- Analog of cycles
- Def. A set S is called *dependent* if not independent
- Def. A dependent set S is a *circuit* if no proper subset is dependent minimally dependent

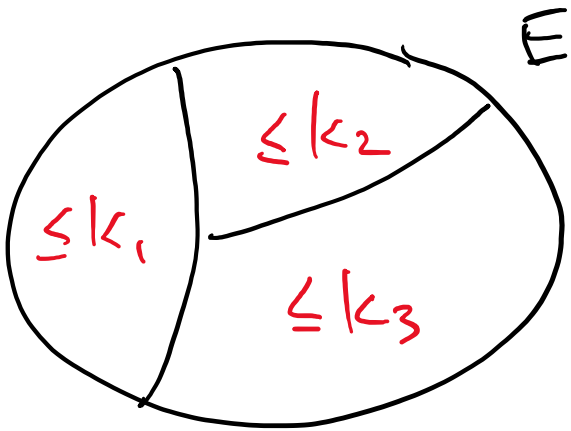
- Obs. A dependent set S is a circuit if and only if every proper subset is independent

- Example: Removing an edge from a cycle yields an acyclic graph

Examples

- Uniform matroid:
  - n = size of ground set
  - k = capacity
  - S independent if $|S| \leq k$
- Partition matroid
  - n = size of ground set
  - Partition of ground set into parts $E_1, \ldots, E_m$
  - Capacities $k_1, \ldots, k_m$





Max Weight Basis Problem

- **Given:** Matroid M and weight function w (can be neg, 0, pos)
- **Goal**: Find max-weight basis B
- Does this capture the Minimum Spanning Tree problem?
- Can you think of a greedy algorithm?

Greedy

- Initialize $B \leftarrow \emptyset$
- For each $e \in E$ (in decreasing order of weight):
  - Add e to B if $B \cup \{e\}$ independent
- Return B

Greedy Optimality for Matroids

Thm. Let M be a matroid. Then for every weight function, Greedy finds the max-weight basis.

Pf Let B = greedy, T = max-weight basis.

① B is a basis

$\Rightarrow |B| = |T|$

Suppose $|B| = |T| = k$

$$B = \{b_1, \dots, b_k\}$$
$$T = \{t_1, \dots, t_k\}$$

decreasing order of weight.

Suppose, towards contradiction, that $w(B) < w(T)$

$B_j = \{b_1, \dots, b_j\}$ $T_j = \{t_1, \dots, t_j\}$.

There exists i s.t. $w(B_i) < w(T_i)$.

$B_{i-1} = \{b_1, \dots, b_{i-1}\}$

$T_i = \{t_1, \dots, t_{i-1}, t_i\}$

$w(t_j) \geq w(t_i) > w(b_i)$

There exists $t_j \in T_i \setminus B_{i-1}$ s.t. $\{t_j\} \cup B_{i-1}$ indep. $\square$

What about Max Weight Independent Set?

Greedy

- Initialize $B \leftarrow \emptyset$
- For each $e \in E$ (in decreasing order of weight):
  - Add e to B if $B \cup \{e\}$ independent
- Return B

Does this also work for independence systems?

Do we know of an NP-hard problem that is a max-weight independence set problem?

Knapsack Problem

$C = 2$, $s(e_i) = 1 \quad \forall i \neq 1$

$s(e_1) = 2$

$w(e_1) = 3$

$w(e_2) = w(e_3) = 2$

- Given items with weights and sizes, and capacity C find max-weight subset with total size at most C
- Can we construct an instance where Greedy is bad?

Greedy always takes heaviest item.

$C = n$ $\quad w(e_1) = 2 \quad s(e_1) = n$

$w(e_i) = 1 \quad s(e_i) = 1$

Greedy $= 2$

OPT $= n - 1$

- Observe: This is an instance in which augmentation does not hold

Greedy Characterization of Matroids

Theorem.

- Let M be an independence system
- M is a matroid if and only if for every weight function, Greedy finds the max-weight independent set

What's it good for?

- Many problems have the form "find max-weight independent set"
- To show Greedy works, show that feasible sets is a matroid
- To show Greedy fails, show that feasible sets not a matroid

Application: Scheduling with Deadlines

Scheduling w/ Deadlines

n jobs

job j has deadline d_j, profit p_j

On each day, can only do 1 job

Each job takes one day.

<u>Goal</u> Find an order to perform jobs to max profit.

<u>Ex</u> 3 jobs

	Deadline	Profit
Job 1	1	5
Job 2	2	10
Job 3	3	1

Reformulation

Def Set of jobs S is feasible if there is some order of jobs in which every job in S is completed on time.

$$\max \quad p(S)$$
$$\text{s.t.} \quad S \in \mathcal{F}$$

$\hookrightarrow$ feasible subsets of jobs.

Lemma Let S be set of job

Def $S(t) = \{ j \in S \mid d_j \leq t \}$.

Then S is feasible if and only if

$$|S(t)| \leq t \quad \forall t.$$

Lemma Let S be set of job

Def $S(t) = \{ j \in S \mid d_j \leq t \}$.

Then S is feasible if and only if

$$|S(t)| \leq t \quad \forall t.$$

Pf Suppose $|S(t)| > t$ for some t

Then there is some job $j \in S(t)$ that is completed after $t \geq d_j$.

Thus, S not feasible.

Suppose $|S(t)| \leq t$ for every t.

Order S in ascending order of deadlines and arbitrarily order remaining jobs.

Every job $j \in S$ completed on day $|S(d_j)| \leq d_j$ $\square$

<u>Lemma</u> If R, S are feasible and $|R| > |S|$, then there exists $j \in R \setminus S$, $R \cup \{j\}$ is feas.

<u>Pf</u> Since $|R| > |S|$, then there exists t^* s.t. $|R(t^*)| \leq |S(t^*)|$

$$|R(0)| = 0 = |S(0)|$$

$$|R| = |R(n)| \qquad |S(n)| = |S|$$

$$|R(t^*)| \leq |S(t^*)|$$

$$|R(t^*+1)| > |S(t^*+1)|$$

So, there exists job $j^* \in R(t^*) \setminus S(t^*)$ w/ deadline t^*+1

Suppose $S' = S \cup \{j^*\}$.

Case 1 $(t \le t^*)$

$$|S'(t)| = |S(t)| \le t$$

Case 2 $(t > t^*)$

$$|S(t)| < |R(t)|$$

$$\begin{aligned} |S'(t)| &= |S(t)| + 1 \\ &\le |R(t)| \\ &\le t. \end{aligned}$$

$\square$